PRML homework 1

KaiWen Zhai

Abstract

Regression analysis is a fundamental task in statistical modeling and machine learning, which aims to explore the quantitative relationship between independent variables and dependent variables. This paper focuses on the comparative study of classical regression models and their optimization strategies, including linear regression solved by least squares, gradient descent and Newton's method, polynomial regression with different orders, and kernel regression based on Gaussian kernel function. Root mean square error and coefficient of determination are adopted as evaluation metrics to quantitatively measure the fitting performance and generalization ability of each model on the training and test sets. Experimental results show that linear regression models with different optimization algorithms obtain consistent performance, but fail to fit the dataset effectively due to the lack of linear correlation in the data. Polynomial regression can capture nonlinear patterns by adjusting the polynomial order, while overfitting will occur when the order is excessively high. Kernel regression achieves the optimal generalization performance by selecting an appropriate bandwidth parameter. This study systematically verifies the adaptability of different regression models to data characteristics, providing a reference for model selection in practical regression tasks.
Keywords: Regression Analysis; Linear Regression; Polynomial Regression; Kernel Regression; Model Fitting; Generalization Performance

1 Introduction

In statistical learning and pattern recognition, regression analysis is a core supervised method for data prediction and relationship fitting, aiming to map input features to output variables for unknown data prediction. Various regression models exist, differing in complexity, fitting ability and efficiency. Linear regression, a simple, interpretable parametric model, can be solved by least squares, gradient descent and Newton's method. It only fits linear relationships, so it performs poorly on nonlinear data. Polynomial regression uses high-order features to fix this but may overfit if too complex. Kernel regression, a non-parametric method, achieves nonlinear fitting via kernel trick, with flexible adaptability to data.

These models lack systematic comparison on the same dataset, especially regarding hyperparameters and optimization algorithms. This paper uses a standard dataset to test linear, polynomial and kernel regression, evaluates their performance, discusses key parameter impacts, and summarizes applicable scenarios for practical use.

2 Methodology

2.1 Metrics

To evaluate the system's predictive ability and its fitting effect on data points, we can use the root mean square to describe the error:

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{1}$$

where y_i denotes the observed value, $\hat{y}_i$ denotes the predicted value of the model, and this formula can be used to characterize the degree to which the predicted values deviate from the observed values. In addition, the coefficient of determination (R^2) can be used to indicate the predictive ability of the model.

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2} \tag{2}$$

We can use this formula to evaluate the goodness of fit. The closer R^2 is to 1, the better the fitting effect. It takes the simplest mean model as the benchmark to assess the fitting performance and reflects the strength of the relationship between variables.

2.2 Linear Regression

Linear regression model is a classical parametric regression model in statistics and machine learning, which is used to characterize the linear dependence between a dependent variable and one or more independent variables. Its core assumption is that the target variable can be approximately represented by a linear combination of independent variables plus a random error term. Its predicted value can be expressed by the formula:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p \tag{3}$$

The equation represents a multiple linear regression model, where y is the response variable, β_0 is the intercept, $\beta_1, \beta_2, \ldots, \beta_p$ are the regression coefficients, and $x_1, x_2, \ldots, x_p$ are the explanatory variables. We have three methods to solve for these coefficients to be determined.

2.2.1 Least Squares

This method obtains the corresponding analytical solution when the loss function is minimized through mathematical calculation, and the parameters of the linear prediction model can be obtained by directly substituting the data points.Given a set of training data points (x_i, y_i) for $i = 1, 2, \ldots, n$, the least squares method aims to find the coefficients $\beta_0, \beta_1, \ldots, \beta_p$ that minimize the cost function:

$$J(\beta) = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2, \tag{4}$$

where $\hat{y}_i$ is the predicted value for the i-th data point, given by the linear model:

$$\hat{y}_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip}. \tag{5}$$

The least squares method can be solved analytically using the normal equation:

$$\beta = (X^\top X)^{-1} X^\top y, \tag{6}$$

where X is the matrix of input features and y is the vector of observed values. This method provides a closed-form solution for the coefficients, making it computationally efficient for small datasets. However, it can be seen from the formula that this method requires matrix inversion, which will lead to excessive computational complexity and low computational efficiency when the data volume is large.

2.2.2 Gradient Descent

Gradient descent (GD) is an optimization algorithm used to minimize the cost function in machine learning models, including linear regression. Given a loss function $J(\theta)$, where θ represents the parameters of the model, the gradient descent algorithm iteratively updates the parameters in the direction of the negative gradient of the cost function with respect to the parameters. The update rule can be expressed as:

$$\theta := \theta - \alpha \nabla J(\theta), \tag{7}$$

where α is the learning rate, and $\nabla J(\theta)$ is the gradient of the cost function with respect to the parameters θ. The key idea behind gradient descent is to take small steps in the direction of the steepest descent of the cost function, which helps to find the optimal parameters that minimize the cost function. In the context of linear regression, the gradient of the cost function with respect to the parameters can be computed as:

$$\nabla J(\theta) = \nabla \left(\frac{1}{n} \|y - X\theta\|^2 \right) = -\frac{2}{n} X^{\mathrm{T}} (y - X\theta), \tag{8}$$

where X is the matrix of input features, y is the vector of observed values, and θ is the vector of parameters. Gradient descent is analogous to descending a mountain: one chooses the direction with the steepest slope, and the negative direction of the gradient is precisely the direction of the fastest descent.

2.2.3 Newton's Method

This method constructs a local quadratic approximation model using the first derivative (gradient) and the second derivative (Hessian matrix) of the function at the iteration point. The iterative update direction is generated by solving the linearized approximation equation, enabling rapid convergence toward the exact solution at a quadratic convergence rate. Given a cost function $J(\theta)$, the Newton's method updates the parameters using the second-order Taylor expansion:

$$\theta := \theta - \left(\nabla^2 J(\theta) \right)^{-1} \nabla J(\theta), \tag{9}$$

where $\nabla J(\theta)$ is the gradient and $\nabla^2 J(\theta)$ is the Hessian matrix of the cost function.Since the linear regression model is essentially a quadratic convex function, the Newton method can be used to quickly find the minimum point of the function.

2.3 Polynomial Regression

Polynomial regression is a type of regression analysis in which the relationship between the independent variable x and the dependent variable y is modeled as an n-th degree polynomial. The polynomial regression model can be expressed as:

$$y = \theta_0 + \theta_1 x + \theta_2 x^2 + \cdots + \theta_n x^n, \tag{10}$$

where $\theta_0, \theta_1, \ldots, \theta_n$ are the parameters of the model. Polynomial regression can capture nonlinear relationships between the independent and dependent variables by introducing higher-order terms of the independent variable. The parameters of the polynomial regression model can be estimated using the least squares method, similar to linear regression, by minimizing the cost function:

$$J(\theta) = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 \tag{11}$$

Polynomial fitting features a simple form and convenient computation. It can approximate data patterns effectively by adjusting the polynomial order, with linearly distributed parameters that are readily solvable via the least squares method. However, excessively high orders are prone to overfitting, sensitive to noise and outliers, and tend to produce severe oscillations in the curve, accompanied by poor extrapolation capability. In the case of high-degree polynomials, additional issues arise including numerical instability and weak interpretability.

2.4 Kernel Regression

The kernel trick is widely applied in machine learning. Its greatest advantage is that it maps low-dimensional data into a high-dimensional space to achieve linear separability while only computing inner products, thereby significantly reducing computational complexity. Kernel regression is a upgraded version of k-nearest neighbors regression(KNN) using kernel tricks to avoid the computational cost of explicit feature mapping. It's a non-parametric method used to estimate the conditional expectation of a random variable. Given a set of training data points (x_i, y_i) for $i = 1, 2, \ldots, n$, the kernel regression estimates the value of the dependent variable y at a new input point x using a weighted average of the observed values y_i

$$\hat{y}(x) = \frac{\sum_{i=1}^{n} K(x, x_i) y_i}{\sum_{i=1}^{n} K(x, x_i)}, \tag{15}$$

where $K(x, x_i)$ is the kernel function that measures the similarity between the input point x and the training data point x_i.

3 Experimental Studies

3.1 Linear Regression

As shown in Table 1 and Fig.1, the performance of three algorithms (least squares, gradient descent, and Newton's method) using metrics defined in Section 2.1 is nearly the same, which is expected since they are all solving the same optimization problem for linear

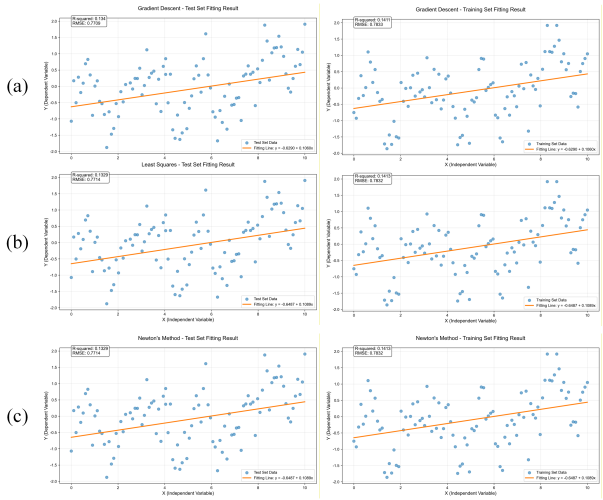

Figure 1: Fitting Curves Under Three Linear Regression Model Solving Methods (a) Gradient Descent (b) Least Squares (c) Newton's Method

Table 1: Performance of three optimal algorithms on fitting linear regression.

Algorithm	Training		Testing	
	RMSE	R^2	RMSE	R^2
Least Squares	0.7714	0.1329	0.7832	0.1413
Gradient Descent	0.7709	0.134	0.7833	0.1411
Newton's Method	0.7714	0.1329	0.7832	0.1413

regression. Note that the performance of gradient descent can be sensitive to the choice of hyperparameters and may require careful tuning and much more time and computational resources to achieve optimal results. The RMSE values are relatively high and the R^2 values are low, indicating that the linear regression model does not fit the data well.

3.2 Polynomial Regression

As shown in Table 2 and Fig., the performance of polynomial regression improves significantly as the degree of the polynomial increases ranging from 1 to 11. But after degree 11, the performance starts to degrade, which may influenced by extreme values in high degrees. It can be seen that overfitting occurs when the order is too high.

Table 2: Performance Analysis of Polynomial Regression with Different Orders.

Order	Training		Testing	
	RMSE	R^2	RMSE	R^2
3	0.7519	0.2086	0.7327	0.2178
7	0.6820	0.3489	0.6807	0.3248
11	0.7714	0.4614	0.6405	0.4023
20	0.8405	0.0110	0.8206	0.0188

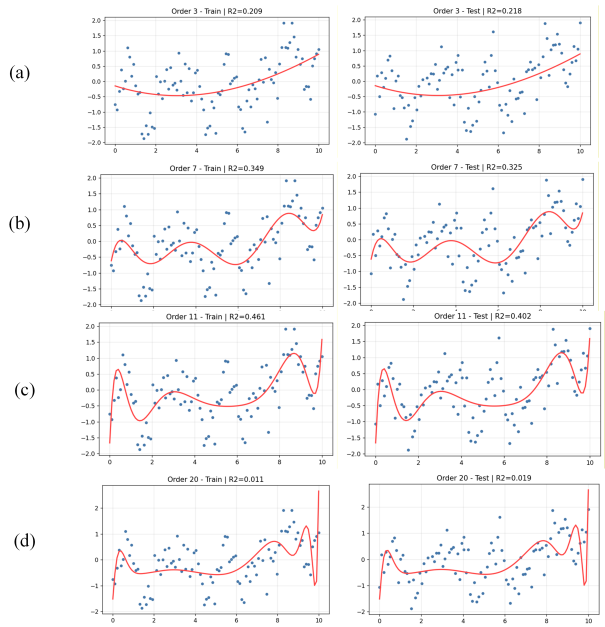

Figure 2: Results of polynomial fitting with different orders

3.3 Kernel Regression

It can be seen that the effect of fitting data using a Gaussian kernel depends on the bandwidth value. When the bandwidth value is small (0.1), the training data can be fitted well, but the generalization ability is poor, indicating overfitting. When the bandwidth is too large (1.5), underfitting can be observed.

Table 3: Performance Analysis of Polynomial Regression with Different Orders.

bandwidth	Training		Testing	
	RMSE	R^2	RMSE	R^2
0.1	0.3178	0.8586	0.5007	0.6347
0.5	0.6004	0.4953	0.6048	0.4670
1.0	0.7165	0.2814	0.7031	0.2795
1.5	0.7451	0.2227	0.7324	0.2183

4 Conclusions

In this paper, we investigate the performance of various regression models and optimization algorithms on a specified dataset. Due to the weak linear characteristics of the data, linear regression cannot effectively describe the relationships between variables. The overall performance of different optimization algorithms is similar, but gradient descent usually requires fine parameter tuning and incurs higher time and computational costs. Polynomial regression with an appropriate order achieves better fitting results, but an excessively high order tends to lead to overfitting. Kernel regression performs the best on the test set, with the smallest root mean square error. In conclusion, the selection of models and optimization algorithms should be based on the characteristics of the data and the inherent relationships between variables.

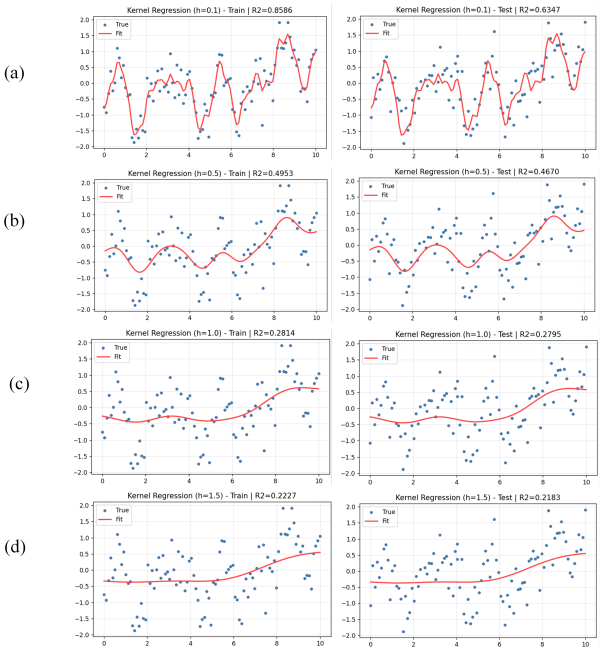

Figure 3: Kernel regression effects under different bandwidths